

Rapport de projet

Ce projet consiste en un programme python permettant de contrôler automatiquement les déplacements d'un robot. Nous avons d'abord codé une simulation avec un affichage fonctionnant avec pygame. Nous avons ensuite codé un traducteur permettant d'utiliser le programme avec l'API du robot.

Mode d'emploi :

Le programme se lance depuis la racine du dossier avec la commande python `sources.main.py`. Tous les paramètres sont optionnels :

- Le premier paramètre détermine si on utilise le robot réel (0) ou la simulation (1)

Les prochains paramètres ne sont utilisés uniquement pour la simulation :

- L'angle (en radian) dans lequel le robot est placé au début de la simulation
- La position x du robot au début de la simulation
- La position y du robot au début de la simulation

Le robot fera alors les actions qui lui sont commandées par le programme jusqu'à la fin. Pour modifier les ordres donnés au robot il faut se rendre dans `controleur` puis dans `control.py`. Les ordres au robot se font par des stratégies.

Il existe 2 stratégies unitaires :

- Une permettant d'avancer en ligne droite à une vitesse et sur une distance donnée.
- L'autre permettant de faire tourner le robot sur lui-même à une vitesse et sur un angle donné.

Et 4 stratégies général qui permettent de combiner d'autres stratégies :

- `Séquence` permet de réaliser à la suite des stratégies mises en paramètre sous forme de liste
- `Strat_while` permet de recréer une boucle while. On met en paramètre la stratégie à effectuer ainsi que la condition du while.
- `Strat_for` permet de recréer une boucle for. On met en paramètre la stratégie à effectuer et le nombre de fois que l'on veut qu'elle s'exécute.
- `Strat_if` permet de recréer une boucle if. On met en paramètre 2 stratégies et une condition, si la condition renvoie true la première stratégie est effectuée, sinon c'est la deuxième.

Pour utiliser les stratégies il suffit d'aller dans `instruction_robot` dans `control.py` et d'ajouter toutes les stratégies qu'on souhaite que le robot réalise dans `tab_strat`.

Schéma fonctionnel :

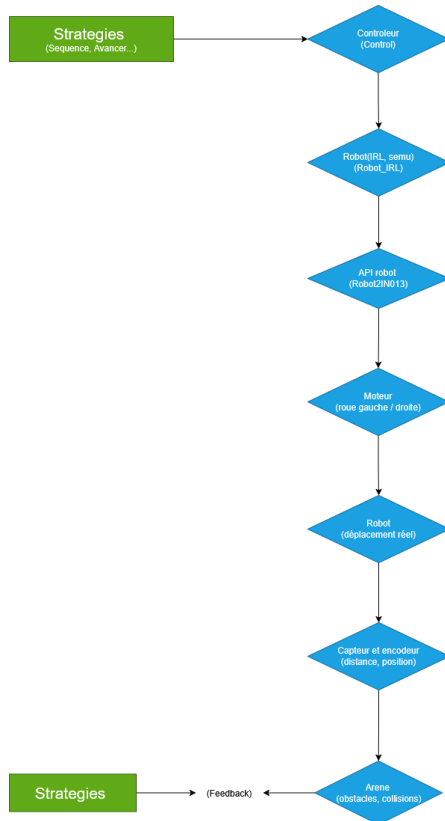
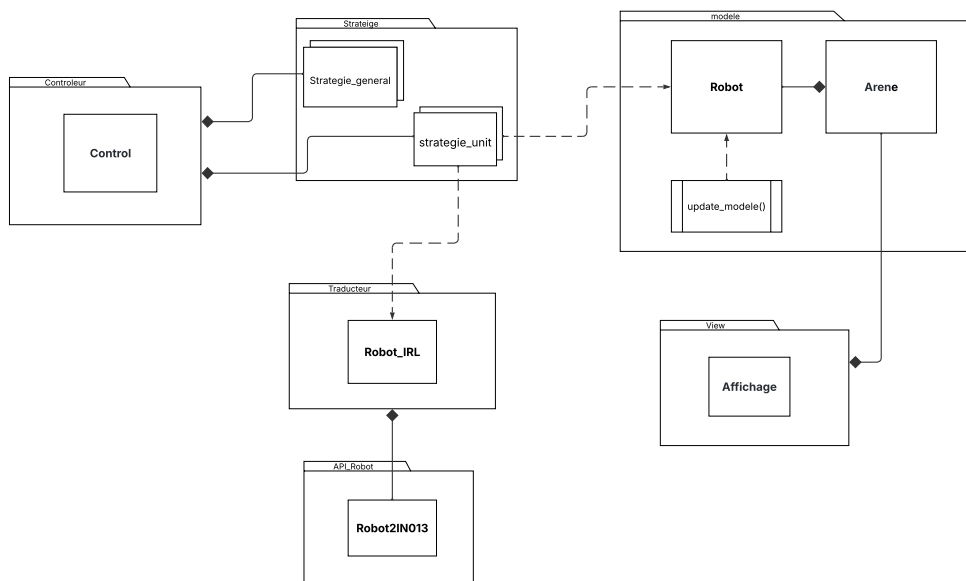


Schéma d'architecture :



Équations de la simulation :

Le robot simulé à une position x et une position y et un angle. La fonction update_pos met à jour ces coordonnées en fonction des vitesses des roues du robot. On a :

$$x = x + v * \cos(\text{angle} + \text{omega})$$

$$y = y + v * \sin(\text{angle} + \text{omega})$$

$$\text{angle} = \text{angle} + \text{omega}$$

Avec :

$$v = (\text{vitesse_d} + \text{vitesse_g}) * \text{ray} / (\text{UPDATE_TIME} * 2)$$

$$\text{omega} = (\text{vitesse_d} - \text{vitesse_g}) * \text{ray} / (\text{UPDATE_TIME} * \text{size})$$

Où :

- Vitesse_d est la vitesse de la roue droite en RAD/s
- Vitesse_g est la vitesse de la roue gauche en RAD/s
- ray est le rayon des roues
- UPDATE_TIME est le temps depuis le dernier appel d'update_pos
- Size est la largeur du robot

Il y a également rot_g et rot_d qui correspondent à l'angle l'angle en radian duquel les roues ont tourné depuis le début de la simulation. Les calculs sont :

$$\text{rot_g} = \text{rot_g} + \text{Vitesse_g} / \text{UPDATE_TIME}$$

$$\text{rot_d} = \text{rot_gd} + \text{Vitesse_d} / \text{UPDATE_TIME}$$

Pistes d'améliorations :

- Permettre de contrôler plusieurs robots simultanément, de les faire interagir. Faire des jeux types chat
- Utiliser la caméra du robot pour détecter des obstacles (la détection d'obstacles étant déjà implémenté dans la simulation).

Groupe : Chef le chat

Membres : Titouan Laville, Dylan Phan Nguyen, Guillaume Dong, Lina Hamlaoui,

Fitia Rahariosa Sendrison, Clément Claeysen